

Enterprise Architecture Reference

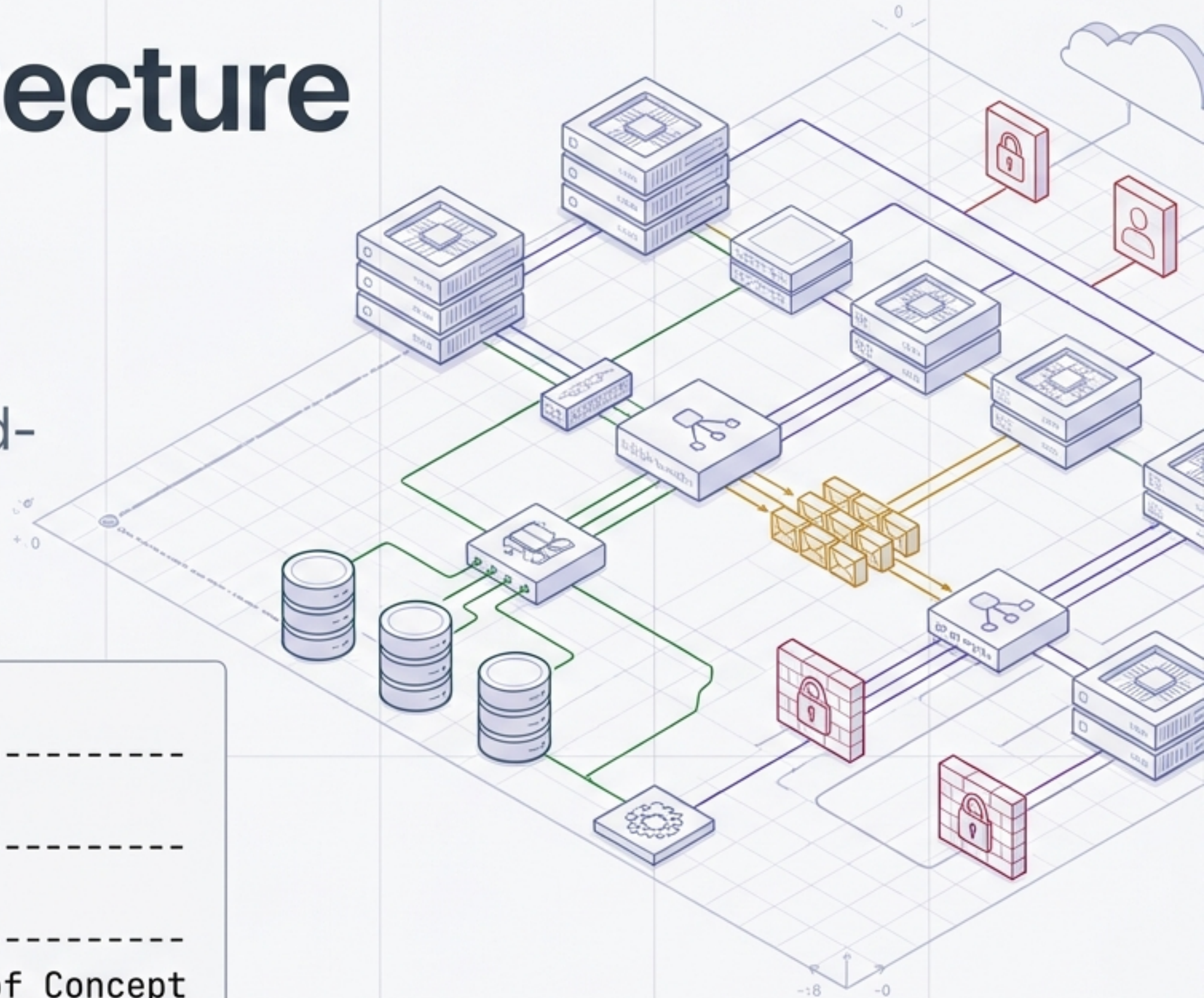
Synthesizing 28+ specialized technical studies into a unified cloud-native engineering blueprint.

Author: k1729p

Language Mastery: Java 25 (LTS)

Lines of Code: 100k+

Format: Interactive Documentation & Proof of Concept



The Knowledge Index

Core Language Mechanics

[Ref: Miscellany]

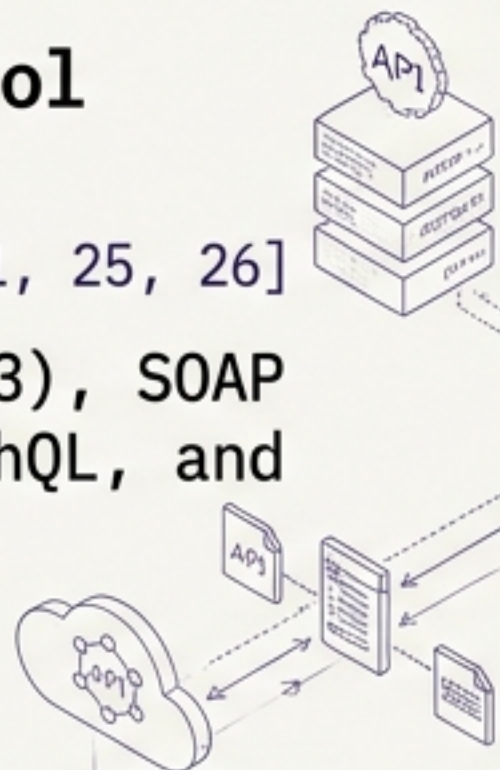
Advanced Concurrency, Reactive Streams, Memory Management, Regular Expressions.



API & Protocol Evolution

[Ref: Study01, 11, 25, 26]

REST (OpenAPI 3), SOAP (JAX-WS), GraphQL, and gRPC (HTTP/2).



Distributed Data & Search

[Ref: Study02, 15, 16]

Reactive MongoDB, Redis In-Memory, Apache Solr, Elasticsearch & Kibana.



Event-Driven Systems

[Ref: Study04, 05, 21]

Apache Kafka Brokers, Spring Cloud Stream, Apache Pulsar Messaging.



Identity & Security

[Ref: Study18, 23, 24]

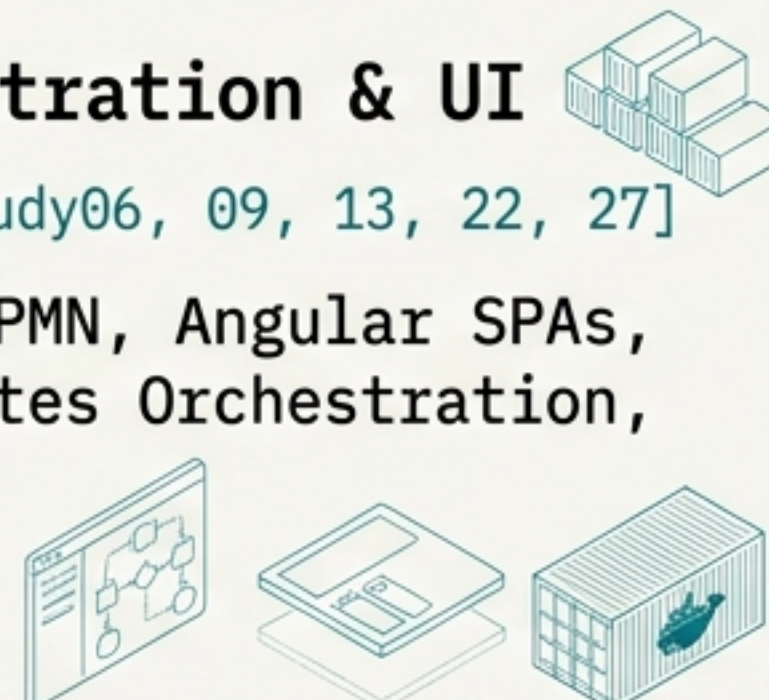
Keycloak IdP, OAuth2 BFF Gateway, JWT, Elliptic-Curve Cryptography.



Orchestration & UI

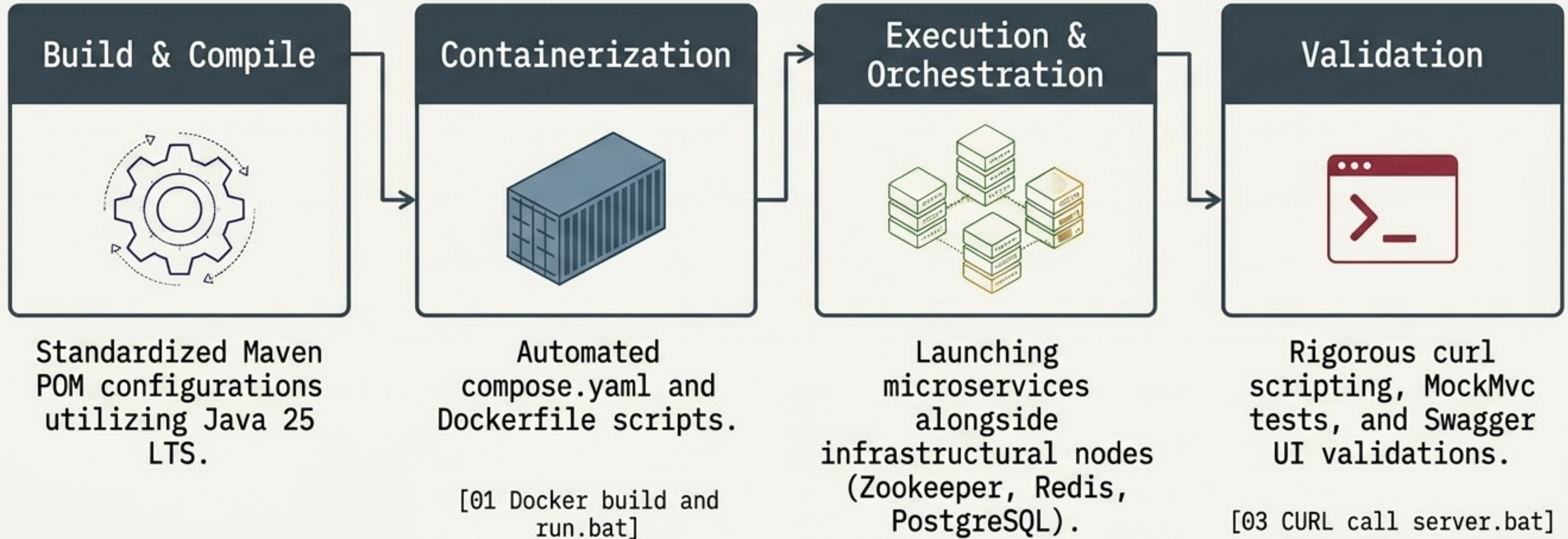
[Ref: Study06, 09, 13, 22, 27]

Zeebe BPMN, Angular SPAs, Kubernetes Orchestration, Docker.



[All modules verified via SonarQube, JUnit 5, Mockito, and Testcontainers]

Developer Experience (DX) & Tooling

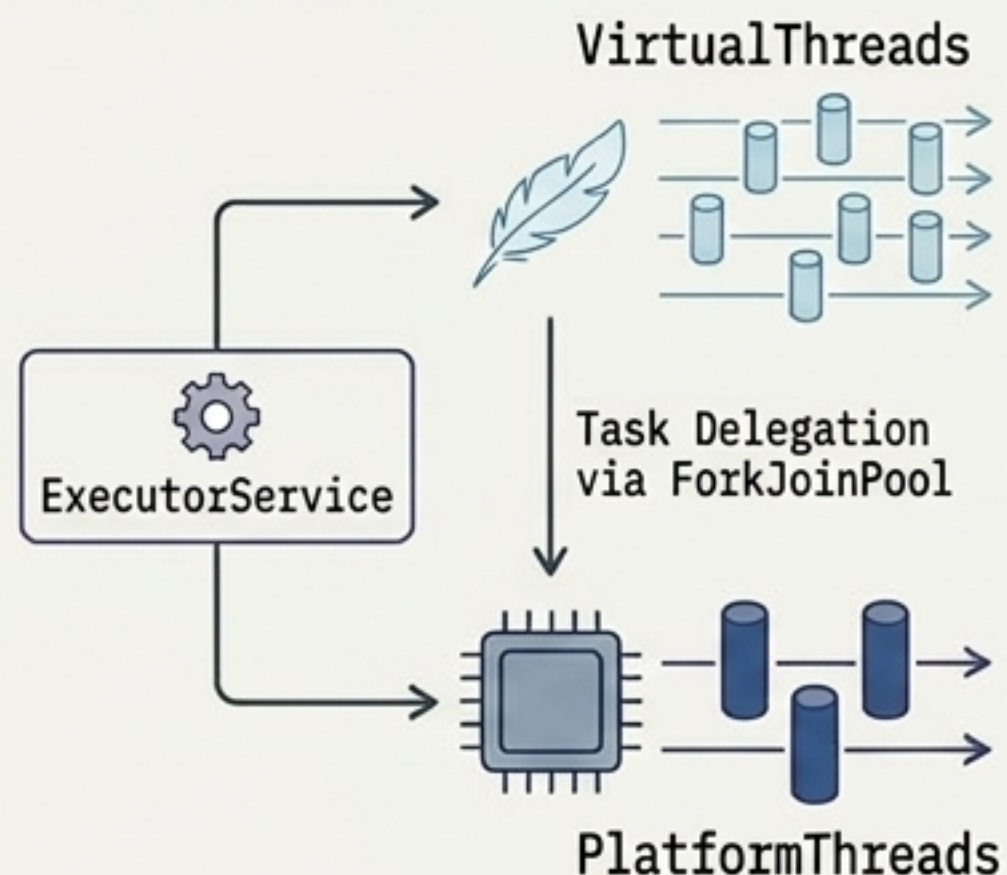


Strict repository traceability maintained via automated batch execution logs and standardized test outputs across all 28 studies.

Core Java Mastery

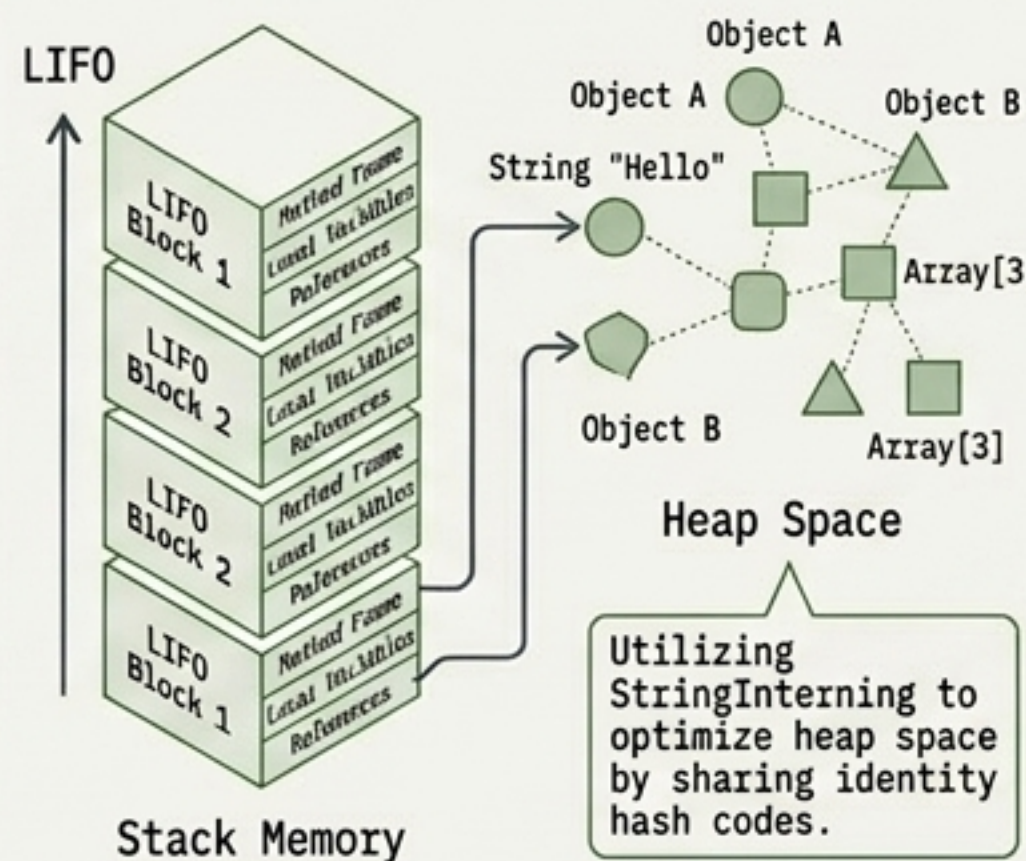
[Ref: Miscellany]

Advanced Concurrency



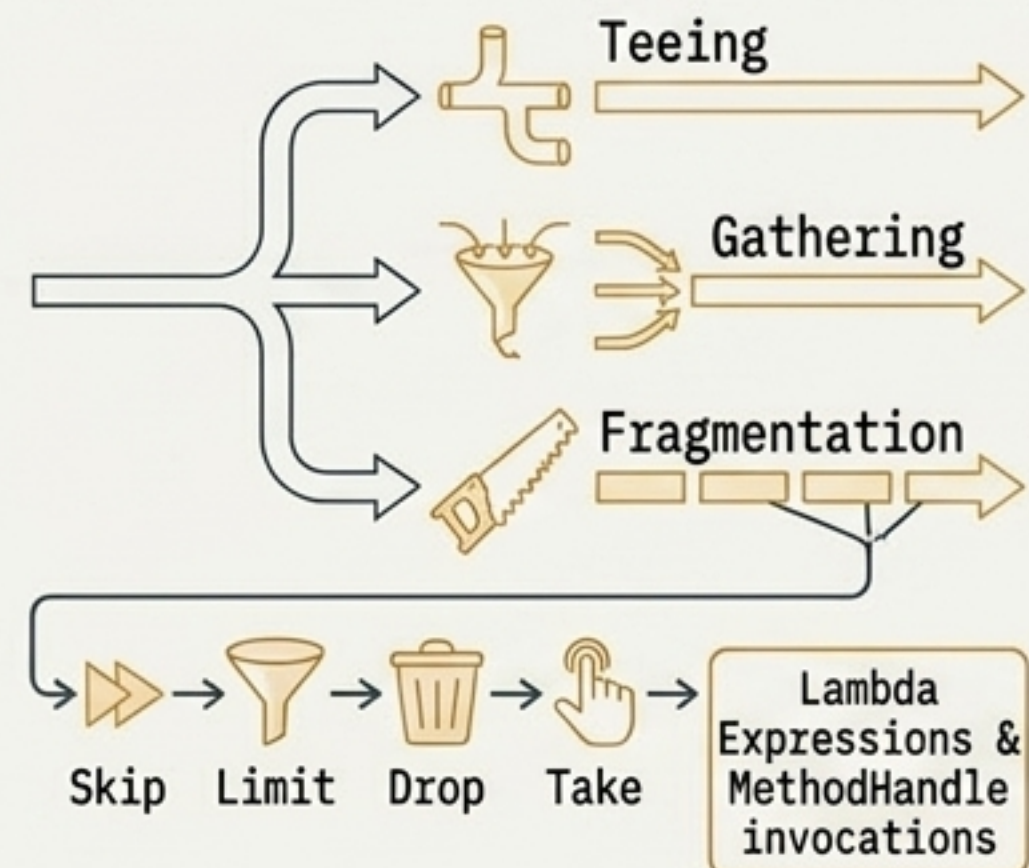
Contrasting lightweight virtual threads for high-throughput with traditional OS-bound platform threads. Task delegation via ForkJoinPool.

Memory & Execution



Low-level memory allocation mechanics. Utilizing **StringInterning** to optimize heap space by sharing identity hash codes.

Stream Data Processing



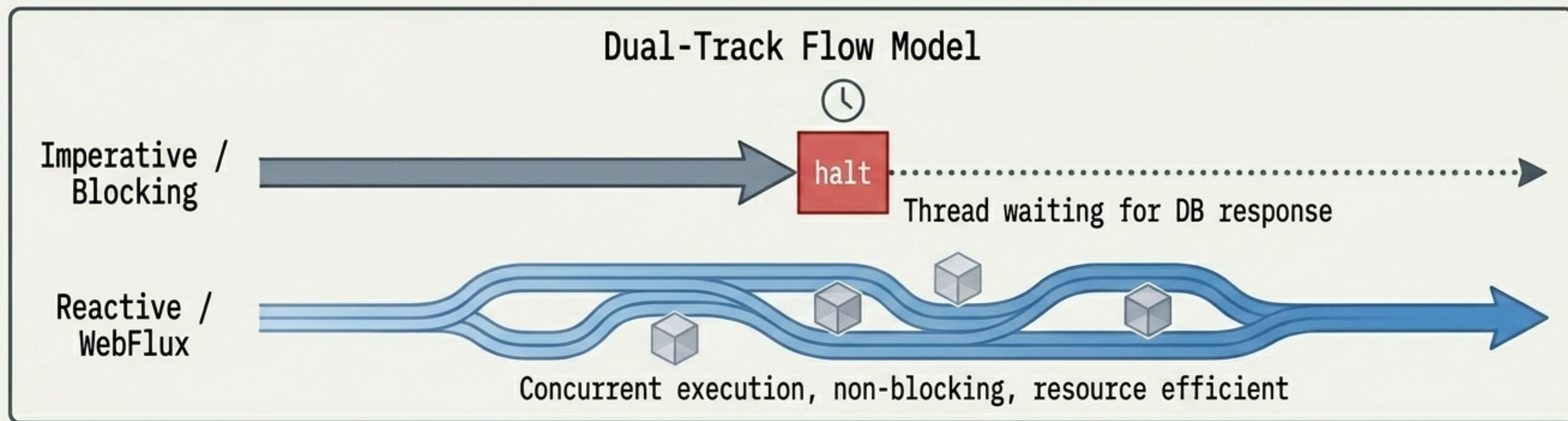
Functional data manipulation (**Skip**, **Limit**, **Drop**, **Take**) using pure Lambda Expressions and MethodHandle invocations.

The API Architecture Matrix

REST (OpenAPI 3.1) [Ref: Study01]	SOAP (JAX-WS) [Ref: Study11]	GraphQL [Ref: Study25]	gRPC [Ref: Study26]
Format: JSON/HAL	Format: XML (WSDL 1.1)	Format: JSON (SDL)	Format: Protobuf (Binary)
Transport: HTTP/1.1	Transport: HTTP/1.1	Transport: HTTP/1.1	Transport: HTTP/2
Model: Stateless, HATEOAS-driven.	Model: Contract-first, highly rigid.	Model: Single-endpoint entity graph, custom @auth directives.	Model: Bidirectional streaming (StreamObserver).
Best For: Standard CRUD, public-facing APIs.	Best For: Legacy enterprise integration.	Best For: Client-driven data fetching.	Best For: High-throughput microservice-to-microservice communication.

Reactive Systems & Data Streams

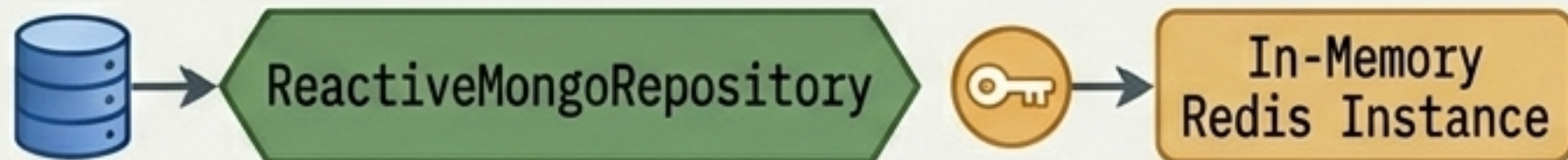
[Ref: Study02, Study03]



The Reactive Core

Explaining `Mono` (0-1 elements, e.g., `findAggregateByDepartmentName`) vs `Flux` (0-N elements, continuous non-blocking stream).

Data Layer Integration

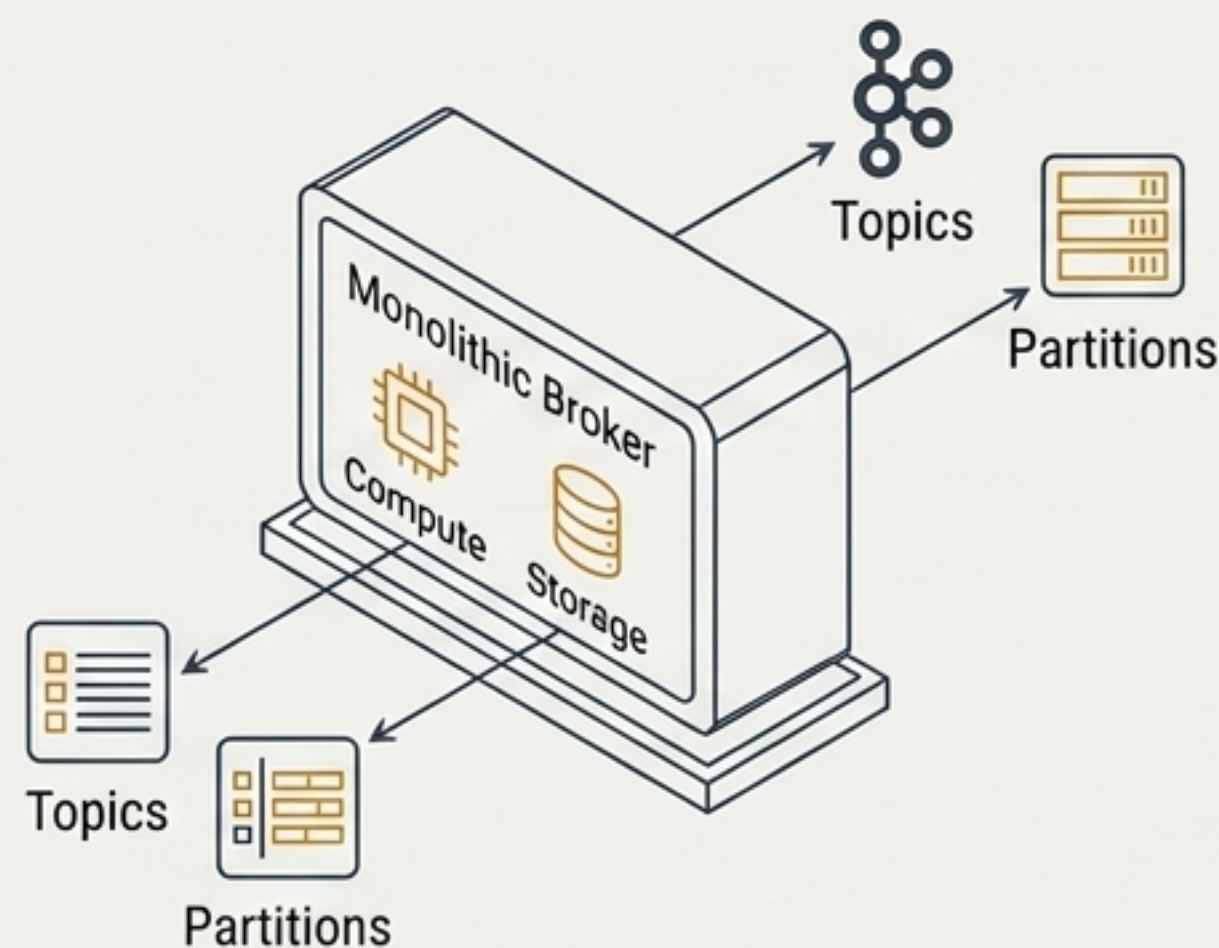


Shift from annotated controllers to functional endpoints to maximize throughput with fewer threads.

Event-Driven Messaging

Apache Kafka

[Study04, Study05]

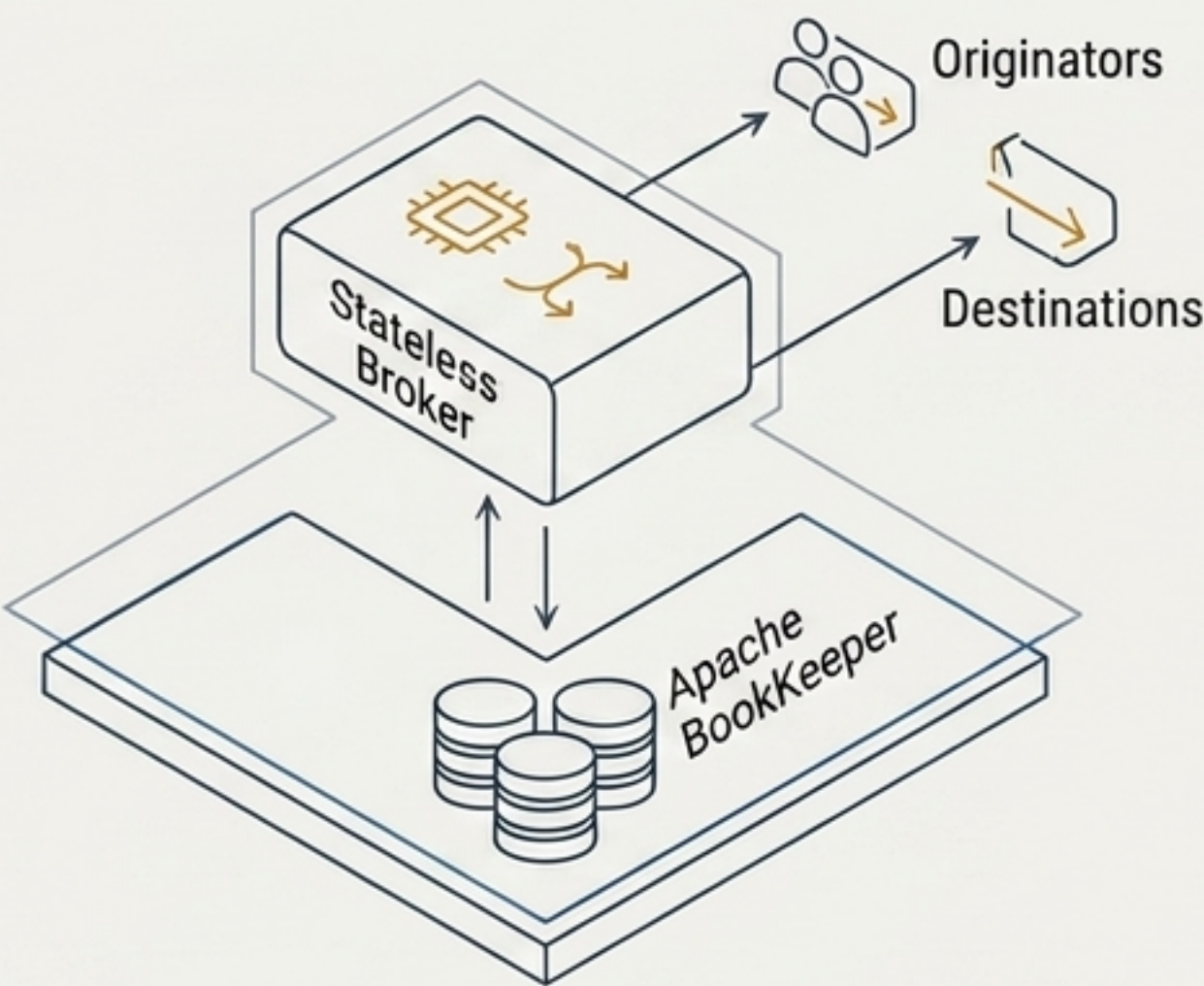


Terminology

Storage/Processing	Data Paradigm	Integration
Monolithic Broker	Topics & Partitions	Spring Cloud Stream & Kafka Binder

Apache Pulsar

[Study21]



Terminology

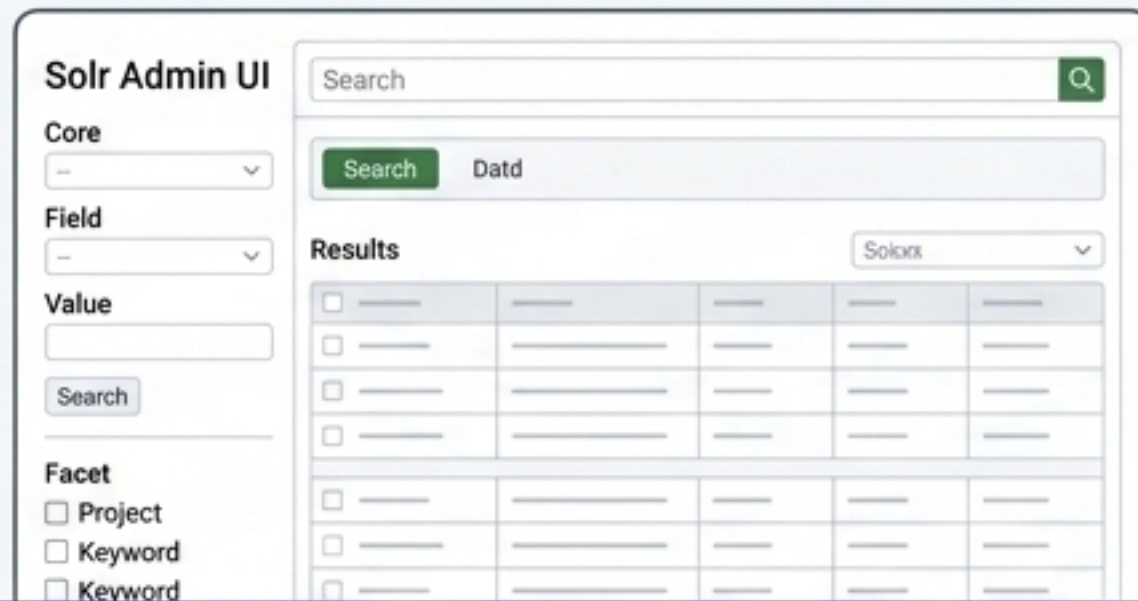
Storage/Processing	Data Paradigm	Integration
Stateless Broker + BookKeeper	Originators & Destinations	Native PulsarClient Reader API

Search & Analytics Engines

Shared Dataset

dataset.json (800 KB)

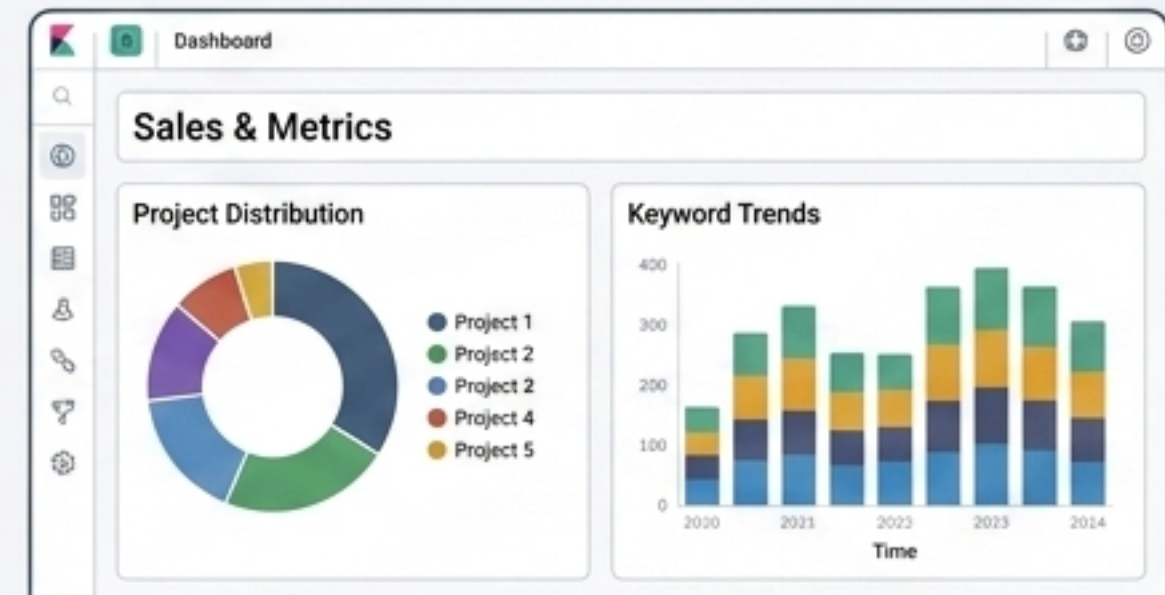
Apache Solr [Ref: Study15]



Query Mechanics: SQL-like syntax in console (e.g., `SELECT DISTINCT project, keyword`).

Strengths: Advanced keyword faceting, Advanced keyword faceting, pivoting, and traditional text analytics.

Elasticsearch & Kibana [Ref: Study16]

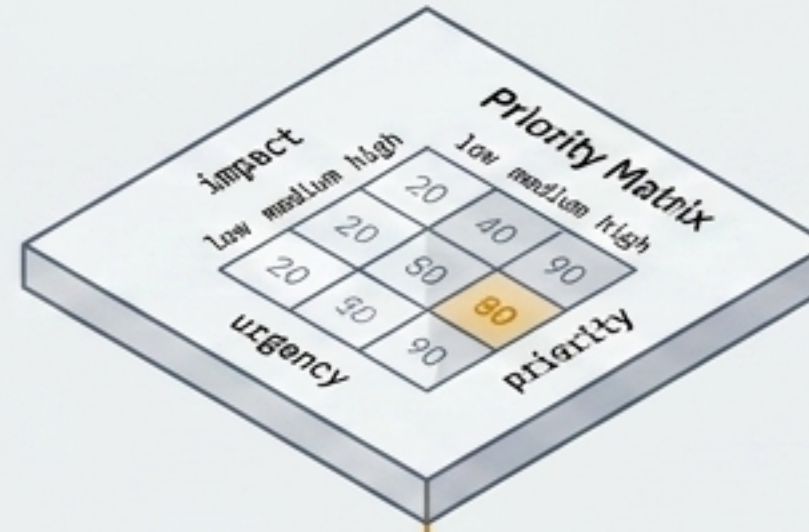


Query Mechanics: JSON-based Match Queries via the Java API Client.

Strengths: Native data visualization and API-key secured REST endpoints.

Workflow Orchestration

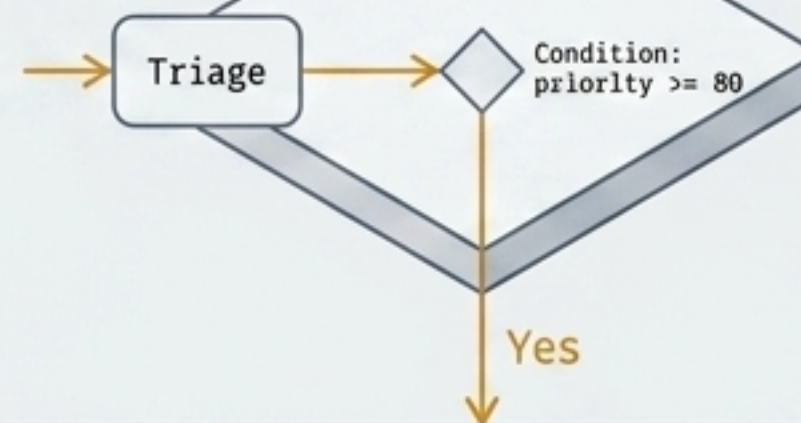
Decision Model
and Notation (DMN)



Priority Matrix

impact					
low	medium	high	low	medium	high
20	20	20	40	40	90
20	50	50	80	80	90
urgency	50	90	priority		

Business Process
Execution (BPMN)



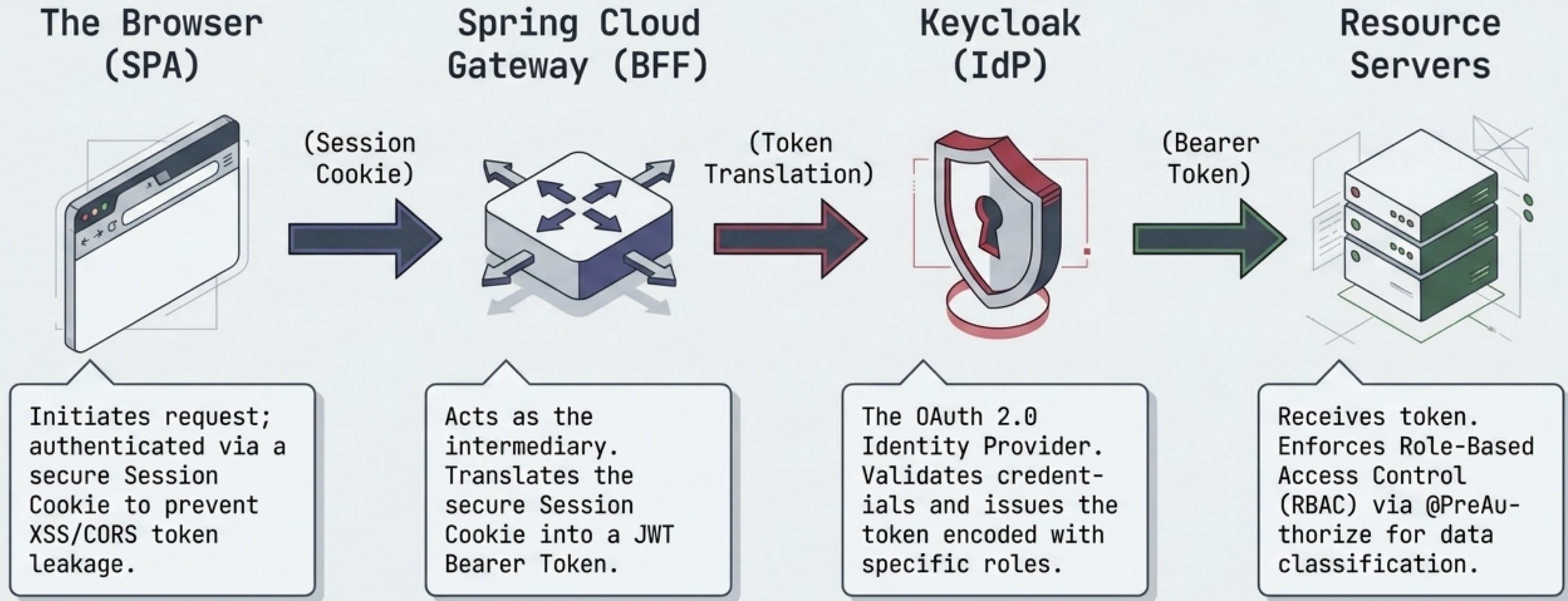
Microservice
Code Translation

The Orchestration Engine ([Camunda/Zeebe](#)) links abstract logic to concrete Java workers.

Task definition 'service-triage' maps directly to `@JobWorker` annotation in `TriageWorker.java`

Task definition 'service-priority-high' maps directly to `HighPriorityWorker.java`

Identity & Access Management (IAM)

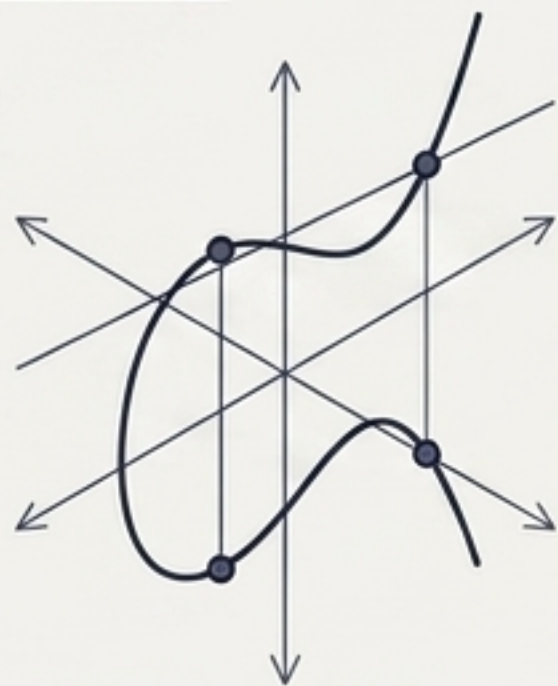


Cryptography & Application Security

[Ref: Study18]

Elliptic-Curve Cryptography (ECC)

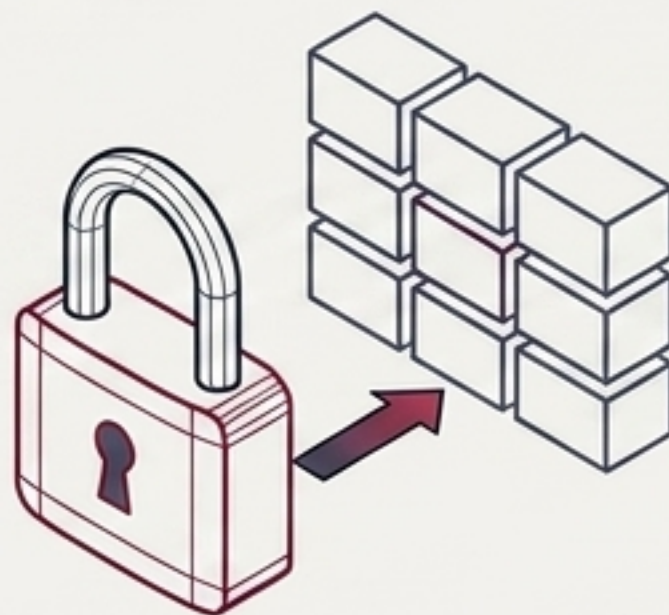
ECDH / ECDSA



Asymmetric key exchange for secure digital signatures, providing equivalent security to RSA with significantly smaller key sizes.

Encryption & Ciphers

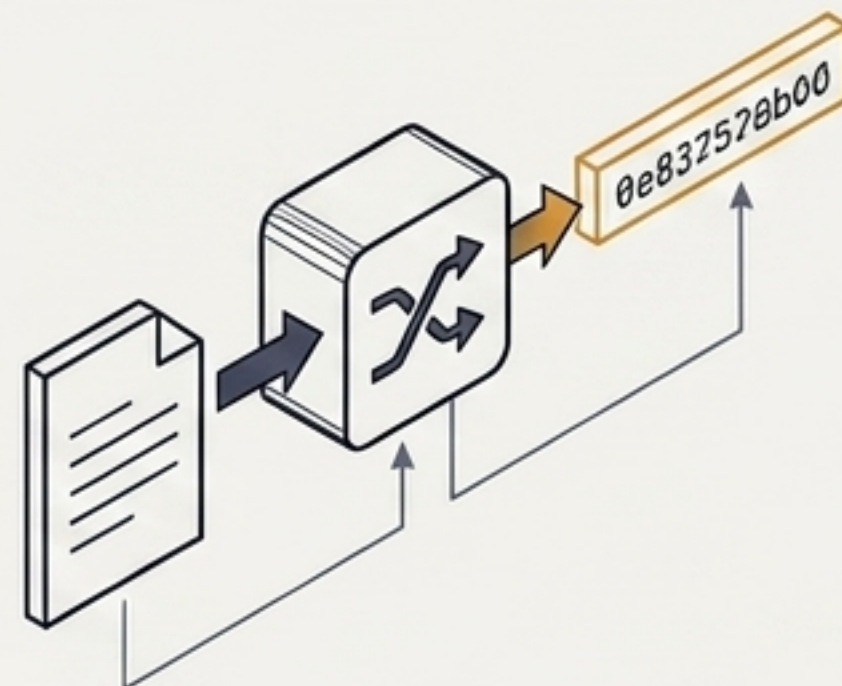
AES-GCM



Symmetric data encryption using Advanced Encryption Standard operating in Galois/Counter Mode for Authenticated Encryption with Associated Data (AEAD).

Integrity & Digests

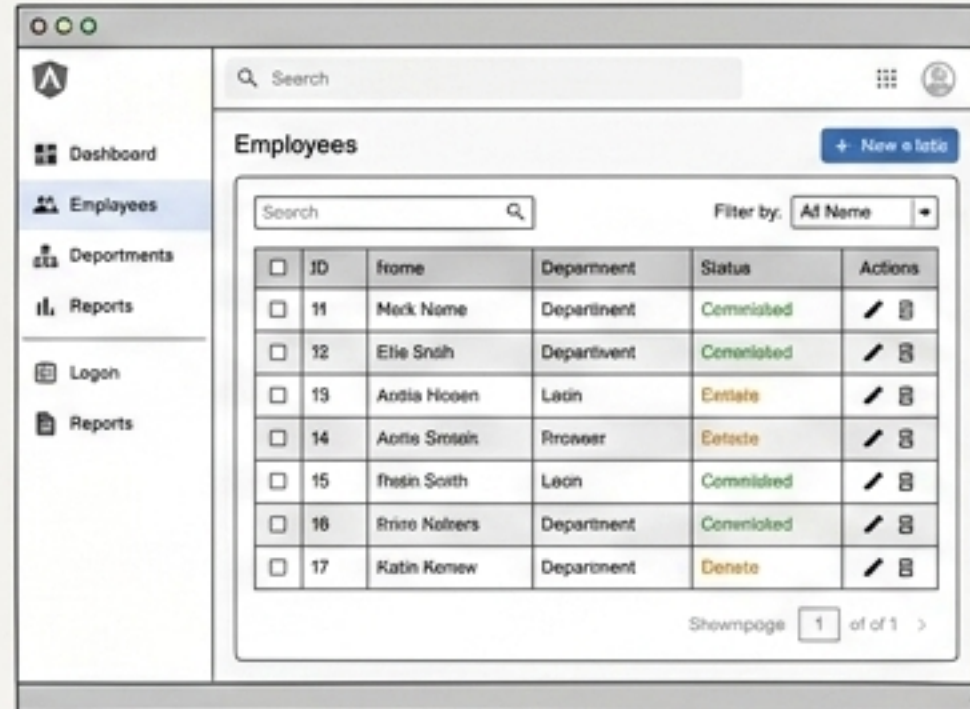
SHA / HMAC



Cryptographic hashing and Message Authentication Codes to guarantee payload immutability and prevent tampering in transit.

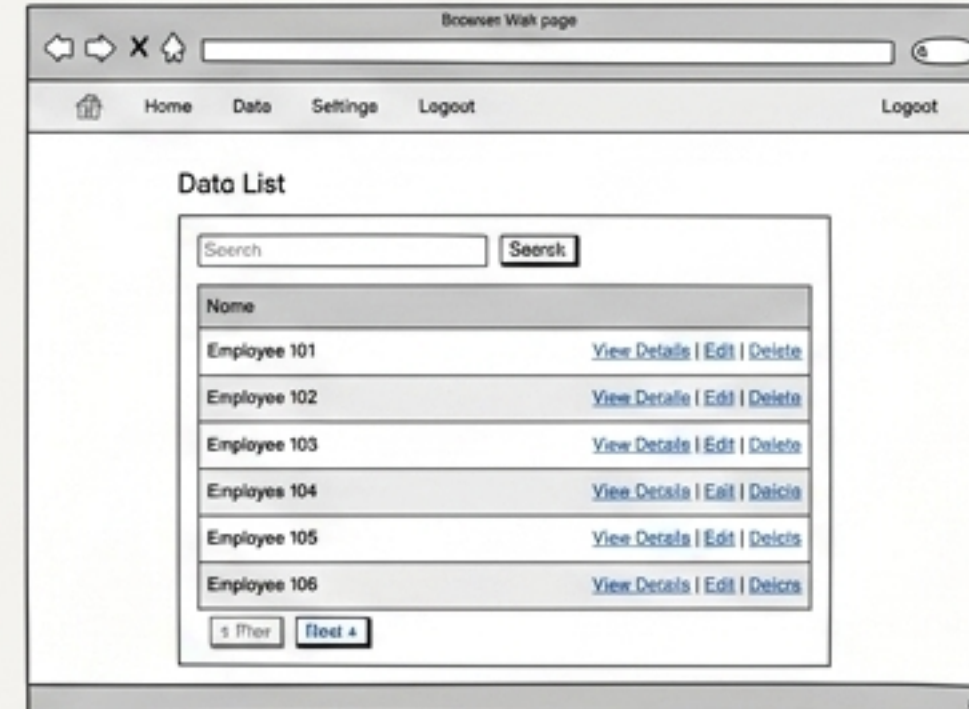
Front-End Integrations

Angular & Material Design



- Client-Side Rendering (CSR)
- Component-driven architecture (DepartmentComponent, _EmployeeComponent, Compsitory).
- End-to-end testing via Cypress; unit testing via Karma.
- PDF generation via pdfmake.

Server-Side Rendering & HATEOAS

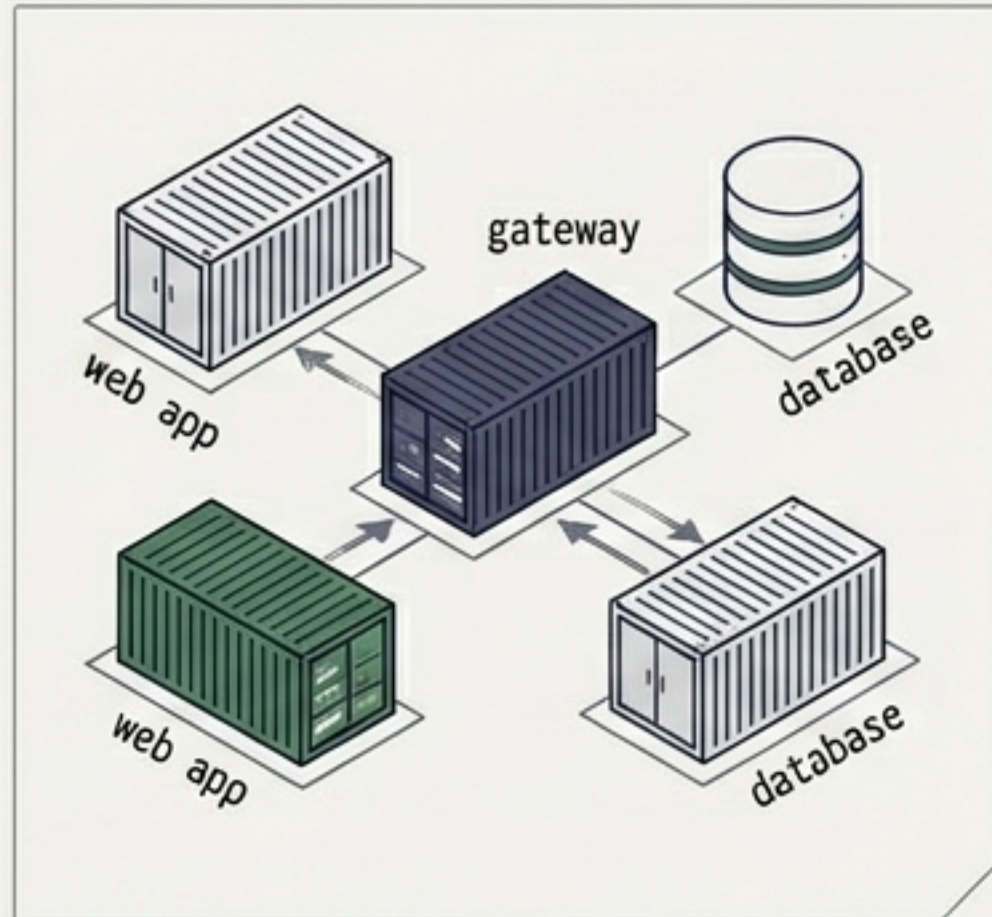


- Spring Web MVC paired with Thymeleaf template engine for traditional stateful session views.
- HATEOAS Integration: Using jQuery/AngularJS to dynamically navigate REST endpoints.
- Driven by Hypertext Application Language (HAL) _links.

Deployment & Containerization

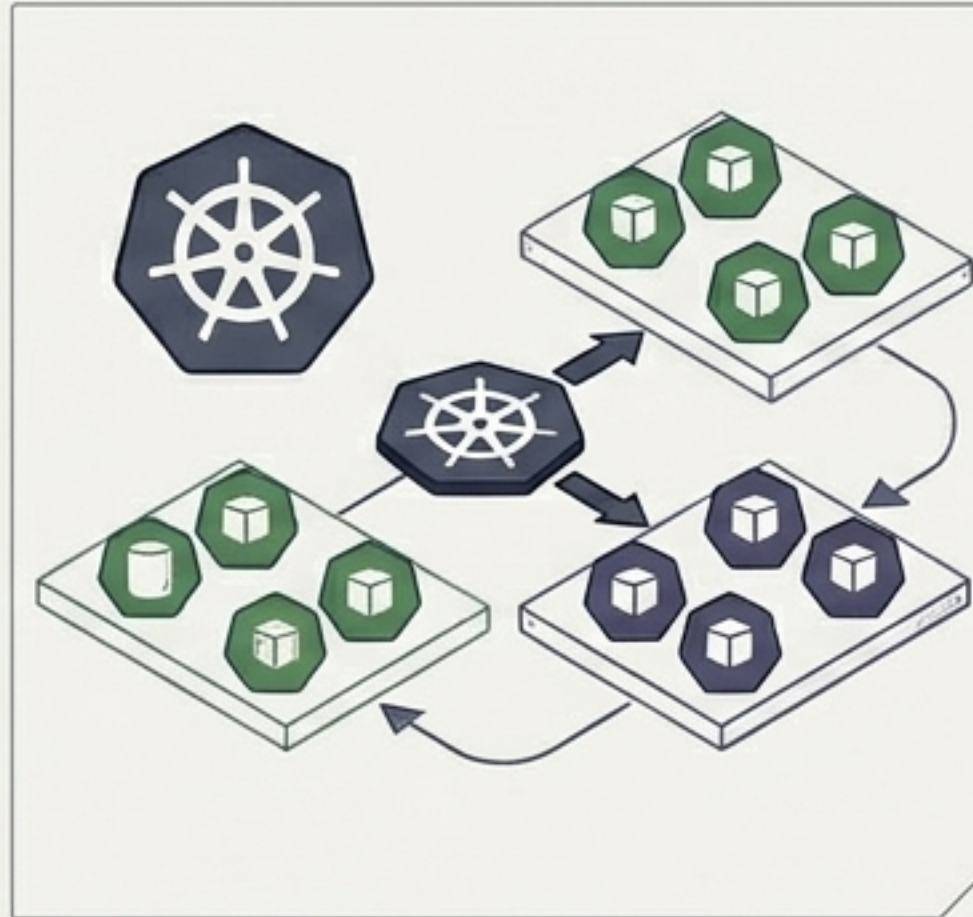
Infrastructure as Code Topology Map

Container Topology



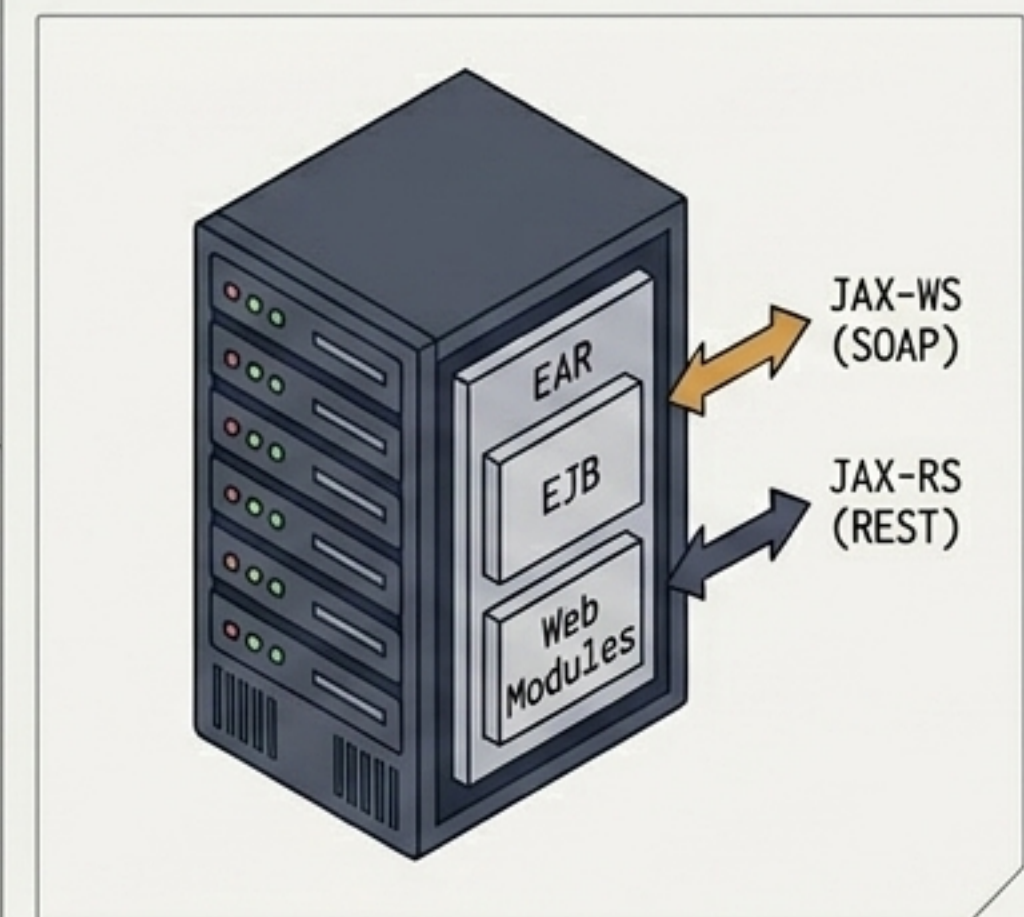
`compose.yaml` setups injecting environment variables into isolated networks linking web apps, gateways, and databases.

Kubernetes Orchestration



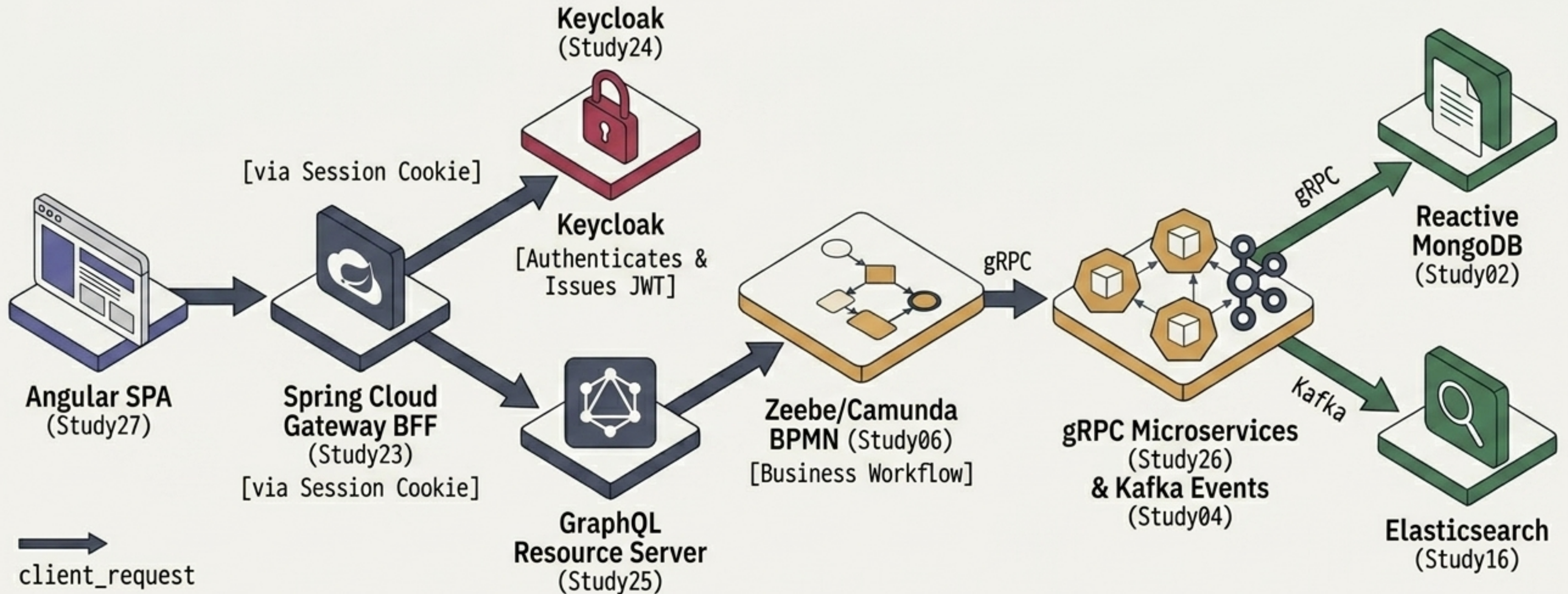
Helm charts and Bitnami images deploying Quarkus Panache applications over PostgreSQL/MongoDB instances inside a 'kind' local cluster.

Enterprise Application Servers



Legacy/Heavy deployment utilizing WildFly/JBoss to host EAR, EJB, and Web modules using JAX-WS (SOAP) and JAX-RS (REST).

SYNTHESIS: The Unified Blueprint



28 distinct repository studies. 1 unified, cloud-native enterprise architecture.

Conclusion & Repository Access

Engineering is about deliberate organization, not just writing code.

Translating scattered READMEs into structured, predictable, and scalable system designs.

Core Philosophy:

- Security-first: Zero-Trust & Keycloak
- Asynchronous by default: WebFlux & Kafka
- Protocol-agnostic: REST, gRPC, & GraphQL

```
> Access the central GitHub Wiki and root index:  
> github.com/k1729p/Portfolio  
> github.com/k1729p/Miscellany  
> exit
```